BACKEND

1. MIGRACIONES

En lugar de crear tablas manualmente en **MariaDB**, escribes código en JavaScript que describe los cambios, lo que permite que todo el equipo tenga exactamente la misma estructura de datos. Crea la estructura de los campos de las tablas (nombre, tipo, valores por defecto, …)

1. ESTRUCTURA DE UNA MIGRACIÓN (UP Y DOWN) - src/database/migrations

- **up**: Define qué debe pasar al aplicar la migración (crear la tabla y sus columnas).
- **down**: Define cómo deshacer esos cambios (normalmente, eliminar la tabla).

2. CÓMO COMPLETAR UN ARCHIVO.JS DENTRO DE LA CARPETA src/database/migrations

Paso 1: Editar el archivo.
Paso 2: Añadir los campos básicos que falten con la siguiente estructura (**up**):

```
up: async (queryInterface, Sequelize) => {
  await queryInterface.createTable('NombreTablas', {
```

Campo con clave principal:

```
id: {
  allowNull: false,
  autoIncrement: true,
  primaryKey: true,
  type: Sequelize.INTEGER
},
```

Campo tipo STRING/TEXT/INTEGER/DOUBLE/FLOAT/DATE/BOOLEAN

```
nombreCampo: {
  allowNull: false,          (omitir si ORM [0..1])
  defaultValue: xxxx,        (valor predeterminado)
  type: Sequelize.TIPO,
  unique: true               (si el valor es único)
  validate: {                // no penalizar en caso de ausencia
      min: 0,
      max: 5
  }
},
```

Campo tipo ENUM

```
nombreCampo: {
  allowNull: false,          (omitir si ORM [0..1])
  type: Sequelize.ENUM,
  values: [                  (ojo, corchete)
     'valor1',
     '……….',
     'valorN'
  ],                         (ojo, corchete)
  defaultValue: 'valor'      (valor predeterminado)
},
```

Paso 3: Añadir, si faltan, los campos de auditoría (**createdAt** y **updatedAt**, obligatorios en Sequalize):

```
Campo createdAt (siempre DATE):
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
```

```
Campo updateAt (siempre DATE):
updatedAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
```

Paso 4: Añadir, si faltan, las claves extranjeras (Foreign Keys), las que conectan la tabla con las demás, según el modelo ORM (las relaciones que salen de la tabla con un *). Ejemplo de estructura:

```
Campo tipo Foreign Keys
nombreCampoId: {
  allowNull: false,
  type: Sequelize.INTEGER,
  references: {
     model: {
        tableName: 'nombreTablaDestino'
   },
   key: 'id'
   },
   onUpdate: 'cascade',                    (si procede)
   onDelete: 'cascade'                     (si procede)
 },
```

Paso 5: Añadir los campos básicos que falten con la siguiente estructura (**down**):

```
down: async (queryInterface, Sequelize) => {
    await queryInterface.dropTable('NombreTablas')
}
```

Paso 6: Abrir el Terminal, situarse en el directorio DeliverUs-Backend y ejecuta las migraciones:

cd DeliverUs-Backend

npx sequalize-cli db:migrate

2. SEEDERS

Los Seeders,"sembradores" son scripts que introducen datos de prueba automáticamente en tu base de datos para que no tengas que escribirlos uno a uno. (carpeta **src/database/seeders**).

- **Introducir** los datos en MariaDB: **npx sequelize-cli db:seed:all**
- **Borra** los datos introducidos por los seeders: **npx sequelize-cli db:seed:undo:all**
- **Automatización**: limpiar, migrar y sembrar todo de una vez:

npm run migrate:backend (desde la raíz del proyecto)

3. MODELOS (El corazón del ORM)

Un Modelo es una clase de JavaScript que representa una tabla de la base de datos. En lugar de usar SQL, usas métodos de esta clase para interactuar con los datos (CRUD). (carpeta **src/models**)

MUY IMPORTANTE: LA TABLA SE NOMBRA EN PLURAL (EJEMPLO: Users) Y EL MODELO EN SINGULAR (User)

1. CÓMO COMPLETAR UN ARCHIVO.JS DENTRO DE LA CARPETA src/models

Paso 1: Edita en esta carpeta el archivo js que corresponde al Modelo modificado en el apartado migraciones.

Paso 2: Incluye las **asociaciones**. Busca el módulo **static associate (models)** dentro de **class nombreModelo extends Model {** {e incluye las relaciones entre Modelos (las Foreign Keys hacia un lado u otro, ojo).

- **belongsTo** (cuando en el ORM la relación con nuestra tabla es * → 1). El modelo que usa belongsTo es SIEMPRE el que guarda la Clave Foránea (Foreign Key) en su tabla.
- **hasMany** (cuando en el ORM la relación con nuestra tabla es 1 → *). El modelo que usa hasMany es el que NUNCA guarda la Clave Foránea (Foreign Key) en su tabla (se guarda la nuestra en la otra tabla).

```
static associate (models) {

    NombreModelo(*).belongsTo(models.ModeloDestino(1), { foreignKey: 'nombreId', as: 'nombreVariableDestino'(igual a ModeloDestino pero la inicial en minúscula), onDelete: 'cascade' (si procede) })

    ……..

    NombreModelo(1).hasMany(models.ModeloOrigen(*), { foreignKey: 'nombreId'(nombreModeloId en minuscula), as: 'nombreVariableOrigenEnPlural' (igual a ModeloOrigen pero en plural) })

    ……..
```

- **belongsTo** (cuando en el ORM la relación con nuestra tabla es 0.1→ *). El modelo que usa belongsTo es SIEMPRE el que guarda la Clave Foránea (Foreign Key) en su tabla.

```
    NombreModelo(*).belongsTo(models.ModeloDestino(1), { foreignKey: 'nombreId', as: 'nombreVariableDestino'(igual a ModeloDestino pero la inicial en minúscula), nullable: true})
```

- **belongsToMany**: Muchos a Muchos (* → *). ¡Necesitamos una tercera tabla intermedia (through) para guardar las claves foráneas de las tablas de los dos Modelos! Habría que modificar los dos modelos de la siguiente forma:

-

```
static associate (models) {

NombreModelo1.belongsToMany(models.NombreModelo2, { through:
'TablaIntermedia',   // <-- El nombre de la tabla intermedia

foreignKey: 'nombreId1',  // <-- El nombre de la Foreign Key de la otra tabla

as: 'nombreVariableModelo2Enplural' })

 }
```

```
static associate (models) {

NombreModelo2.belongsToMany(models.NombreModelo1, { through:
'TablaIntermedia',   // <-- El nombre de la tabla intermedia

foreignKey: 'nombreId2',  // <-- El nombre de la Foreign Key de la otra tabla

as: 'nombreVariableModelo1Enplural' })

 }
```

- Paso 3: Incluye los **atributos**. Busca el siguiente módulo (donde **nombreModelo** es el nombre del Modelo que has modificado en migraciones), e incluye cada uno de los campos básicos, los campos de auditoría y las Foreign Keys que has creado (ojo, los campos de auditoria (createdAt y updatedAt) también se incluyen aquí), pero la primaryKey (ID) no se incluye:

```
nombreModelo.init({
   campo: DataTypes. STRING/TEXT/INTEGER/DOUBLE/FLOAT/DATE/BOOLEAN,
            allowNull: false        (si procede)
            unique: true            (si procede)
   ……:
   campoEnum: DataTypes.ENUM('valor1, '……', 'valorN'),
   campoForeignKey campoId: DataTypes.INTEGER,
  }, {
   sequelize,
   modelName: 'nombreModelo'
  })
  return nombreModelo
 }
```

Paso 4: Definir, si existen, **métodos que realicen cálculos** sobre el modelo. Incluir, entre **static associate (models) {….} y nombreModelo**.init({}, los métodos. Por ejemplo:

```
async getAverageServiceTime () {

  try {
    const orders = await this.getOrders()
    const serviceTimes = orders.filter(o => o.deliveredAt).map(o =>
moment(o.deliveredAt).diff(moment(o.createdAt), 'minutes'))
    return serviceTimes.reduce((acc, serviceTime) => acc + serviceTime, 0) / serviceTimes.length
  } catch (err) {
    return err
  }

 }
```

4. RUTAS (ROUTES)

- **Rutas (routes):** Definen la URL y el método HTTP (GET, POST, PUT, DELETE).
- **Controladores (controllers):** Contienen la lógica que interactúa con la base de datos.

El software backend puede publicar sus funcionalidades a través de los servicios RESTFul. Estos servicios siguen los patrones arquitectónicos del protocolo HTTP.

CRUD (CREAR (**C**REATE) – LEER (**R**EAD) – ACTUALIZAR (**U**PDATE) – ELIMINAR (**D**ELETE))

Si el sistema proporciona operaciones CRUD sobre una entidad, debería haber **un punto final** (**endpoint**) para cada operación:

Acción	método HTTP	Ejemplo	Controlador
Crear	POST	Crear un restaurante	create
Leer (todos)	GET	Leer todos los restaurantes	index
Leer (un parámetro de ruta)	GET	Leer detalles restaurante id	show
Actualizar (un parámetro de ruta)	PUT / PATH	Actualizar detalles restaurante id	update
Eliminar (un parámetro de ruta)	DELETE	Eliminar el restaurante id	destroy

PUT: Actualiza todos los campos del recurso; PATH: Para Actualizar uno o varios campos (no todos).

Además, y el punto final puede definir algunos parámetros de consulta. Normalmente, estos se destinan a incluir algunos parámetros opcionales en la solicitud, como implementar una búsqueda sobre la entidad. Por ejemplo, si queremos consultar los pedidos filtrados por estado, debe definirse un parámetro de consulta. El método de controlador es: **status.**

DEFINICION DE RUTAS - PLANTILLA- **src/routes/archivo.js**

```
app.route('/path') //la ruta del punto final (endpoint)
   .get( //el método HTTP que queremos esté disponible en la ruta anterior
    EntityController.index) // controlador que atenderá las solicitudes para ese método http y ruta
   .post( // para encadenar más métodos HTTP para el mismo endpoint
    EntityController.create) // controlador que atenderá las solicitudes para ese método http y ruta
```

Ejemplo en archivo: src/routes/RestaurantRoutes.js

```
import RestaurantController  from ('../controllers/RestaurantController.js')

const loadFilesRoutes = function (app)   {

  // Rutas para la colección completa y creación
 app.route('/restaurants')
     .get(RestaurantController.index)  // FR1: Listar restaurantes
     .post(RestaurantController.create) // FR3: Crear restaurante

 // Rutas para un restaurante específico (usando restaurantId como parámetro)
 app.route('/restaurants/:restaurantId')
      .get(RestaurantController.show)    // FR2: Detalle y menú
      .put(RestaurantController.update)  // FR3: Actualizar
      .delete(RestaurantController.destroy) // FR3: Eliminar
}
export default loadFilesRoutes
```

5. MIDDLEWARES Y MIDDLEWARE DE VALIDACION

MIDDLEWARES (/src/middlewares)

Archivo/Funciones	Propósito
AuthMiddleware.js.	Autenticación/autorización. Dos funciones:
isLoggedIn	*Comprueba si el usuario ha iniciado sesión (la solicitud incluye un token portador válido).*
hasRole	*Recibe una serie de nombres de roles y comprueba si el usuario iniciado sesión tiene el rol necesario.*
EntityMiddleware.js	Para comprobar si un ID dado identifica un registro de una entidad determinada en la base de datos. Una función:
checkEntityExists	*Comprueba, para un ID y entidad dados, si existe un registro en la tabla correspondiente de la base de datos que coincida con dicho id y, en caso de que el registro no exista, devuelve el código de estado HTTP 404.*
ValidationHandlingMiddleware.js	Una función:
handleValidation	*Comprueba el resultado de express-validator y, si se encuentra un error, devuelve 422 (error de validación) y detiene el procedimiento de validación.*
FileHandlerMiddleware.js	Una función:
handleFilesUpload	*Recibe un array de nombres de campos, correspondientes a atributos de tipo de archivo para una entidad, así como la ruta base del servidor donde deben almacenarse estos archivos. Es responsable de gestionar la subida de los archivos al servidor, incluyendo la generación de un nombre único para cada archivo y el almacenamiento de las rutas completas en los campos correspondientes de la entidad.*

Además, existen Middlewares por cada entidad: ej. OrderMiddleware.js, ProductMiddleware.js, RestaurantMiddleware.js…

Para comprobar todos estos requisitos, tenemos que incluir cada método de middleware en la ruta correspondiente (en la carpeta src/routes):

PLANTILLA: /src/routes/archivo.js

```
app.route('/path') //la ruta del punto final (endpoint)
  .post(
    isLoggedIn,
    hasRole('owner'),
    checkEntityExists (Restaurant,'restaurantId')
    handleFilesUpload(['image'], process.env.PRODUCTS_FOLDER),
    ProductValidation.create,
    handleValidation,
    ProductMiddleware.checkProductRestaurantOwnership, (*)
    ProductController.create
  )
```

(*) Si se han importado los middlewares, no hace falta poner EntidadMiddleware.

EJEMPLOS DE ESTRUCTURAS EN LA CARPETA MIDDLEWARE

Estructura base de un middleware

Siempre recibe req, res, next

```
const miMiddleware = async (req, res, next) => {
try {
// logica de comprobacion
if (condicionOK-debe incluir el res) {
return next() // continua al siguiente middleware
} return res.status(403).send('No tienes permiso') // corta la cadena
} catch (err) {
return res.status(500).send(err)
}
}
```

Regla: Regla de oro: o llamas a **next**() o devuelves una respuesta. Nunca las dos cosas, nunca ninguna

Cómo crear un middleware de ownership

Comprueba que el recurso pertenece al usuario logueado

```
// src/middlewares/EntidadMiddleware.js
import { Entidad } from '../models/models.js'
const checkEntidadOwnership = async (req, res, next) => {
try {
const entidad = await Entidad.findByPk(req.params.entidadId)
if (req.user.id === entidad.userId) {
return next() // es suya
}
return res.status(403).send('Not enough privileges. This entity does not belong to you')
} catch (err) {
return res.status(500).send(err)
}
}
export { checkEntidadOwnership }
```

COMO CREAR UN MIDDLEWARE DE RESTRICCION DE NEGOCIO

Ej: no borrar si tiene pedidos

```
const restaurantHasNoOrders = async (req, res, next) => {
try {
const count = await Order.count({
where: { restaurantId: req.params.restaurantId }
})
if (count === 0) {
return next() // sin pedidos
}
return res.status(409).send('Some orders belong to this restaurant.')
} catch (err) {
return res.status(500).send(err.message)
}
}
```

3.1 MIDDLEWARES DE VALIDACION

Los middlewares de validación están destinados a comprobar si los datos que llegan a una solicitud cumplen con los requisitos de información. La mayoría de estos requisitos se definen a nivel de base de datos y se incluyeron al crear el esquema en los archivos de migración. Algunos otros requisitos se comprueban en la capa de aplicación. Por ejemplo, si quieres crear un nuevo restaurante, se pueden proporcionar algunas imágenes: imagen del logo e imagen del héroe. Estos archivos deberían ser imágenes y su tamaño debe ser inferior a 10 mbs. Para comprobar estos otros requisitos, usaremos el paquete **express-validator.**

Es buena práctica realizar una validación completa usando express-validator, independientemente de si dicha validación está parcialmente incluida en la base de datos o no.

Fíjate en que crearemos una serie de reglas para cada punto final que requeriría validación, normalmente una serie de reglas **create** para crear nuevos datos y otra **update** para actualizar datos.

3.2 DEFINICIÓN DE MIDDLEWARES Y MIDDLEWARES DE VALIDACIÓN PARA RUTAS DE RESTAURANTES

Para cada ruta deberías determinar si:

- ¿Es necesario que un usuario esté conectado? -> Sí: isLoggedIn No: no se incluye
- ¿Es necesario que el usuario tenga un rol concreto? -> hasRole(‘**owner’)** o (‘**customer’**) y si es ambos no se incluye este código.
- hay que comprobar si existe entidad (404) -> checkEntityExists (**Restaurant**, ‘**restaurantId’**)
- ¿Pueden los datos incluir archivos?

  -> Sí: handleFilesUpload([‘image’], process.env.CARPETA) No: no se incluye
- ¿Es necesario que el restaurante pertenezca al usuario registrado? (los datos del restaurante deben incluir un userID que pertenezca al propietario de ese restaurante)-> Si: RestaurantMiddleware.checkRestaurantOwnership No: No se incluye
- ¿Es necesario que los datos del restaurante incluyan valores válidos para cada propiedad para que se creen según nuestros requisitos de información?

  RestaurantValidation.create (o update) OJO: LAS VALIDACIONES NO SE APLICAN A GET Y DELETE
  HandleValidation OJO: LAS VALIDACIONES NO SE APLICAN A GET Y DELETE

3.3. IMPLEMENTAR MIDDLEWARE DE VALIDACIÓN PARA CREAR RESTAURANTE

controllers/validation/archivo.js

Encontrarás los arrays de reglas para validar datos al crear create y actualizar update.

Formatos:

check('name').exists().isString().isLength({ min: 1, max: 255 }).trim(),

exists(): si allowNull: false (si no es .optional())

isString() o según el tipo (isFloat()), isInteger({min: X, max:Y)}

isLength ({min: X, max: Y}) para la medida en caracteres del campo (X=Y para valor fijo)

.trim() para quitar espacios

.custom() si no está en la lista: ejemplo:

```
check('startTime').exists().withMessage('Start time is required')
 .custom(validateTimeFormat),

check('endTime').exists().withMessage('End time is required')
.custom(validateTimeFormat).custom(validateEndTimeAfterStartTime)
```

INCLUIR TAMBIEN EN UN CHECK CON .custom LAS VALIDACIONES AUXILIARES DE ESTE ARCHIVO

Tipo de campo	Regla
Texto obligatorio	.exists().isString().isLength({min:1, max:255}).trim()
Texto opcional	.optional({nullable:true, checkFalsy:true}).isString().trim()
Decimal (precio)	.exists().isFloat({min:0}).toFloat()
Entero / FK	.exists({checkNull:true}).isInt({min:1}).toInt()
Email opcional	.optional({nullable:true,checkFalsy:true}).isString().isEmail().trim()
URL opcional	.optional({nullable:true,checkFalsy:true}).isString().isURL().trim()
Campo NO permitido	.not().exists()

VALIDACION DE FICHEROS CON IMÁGENES

Formatos para campos de archivos a subir con limitación de tamaño (dos check por cada imagen, uno para subirla y otro para limitar el tamaño):

```
check('heroImage').custom((value, { req }) => {

  return checkFileIsImage(req, 'heroImage')

 }).withMessage('Please upload an image with format (jpeg, png).'),

check('heroImage').custom((value, { req }) => {

  return checkFileMaxSize(req, 'heroImage', maxFileSize)

 }).withMessage('Maximum file size of ' + maxFileSize / 1000000 + 'MB'),
```

3.4. COMPROBAR LA VALIDACIÓN EN CONTROLADORES

Cuando falla la validación, se pasa al siguiente método de middleware en la cadena de middleware. En este caso, el siguiente método debe ser el método del gestor de validación, que siempre se ejecutará después del middleware de validación.

```
const handleValidation = async (req, res, next) => {
  const err = validationResult(req)
  if (err.errors.length > 0) {
    res.status(422).send(err)
  } else {
    next()
  }
}
```

6. CONTROLADORES (CONTROLLERS)

Los controladores son los componentes principales de la capa de lógica de negocio. Cada método controlador recibe una solicitud (**req**) y un objeto de respuesta (**res**).

El objeto de solicitud *representa la petición HTTP y tiene propiedades para la cadena de consulta de la petición, parámetros, cuerpo, cabeceras HTTP, etc*

Tipos de solicitudes en este proyecto:

Solicitud	Ejemplo
req.body	representa los datos que provienen del cliente
req.params	representa los parámetros de camino. Por ejemplo, si definimos un parámetro: restaurantId, tendremos acceso a él mediante: **req**.params.restaurantId
req.query	representa parámetros de consulta. Por ejemplo, si una solicitud incluye un parámetro de consulta: status, será accedida por: **req**.query.status
req.user	representa al usuario iniciado sesión que hizo la solicitud.

El objeto respuesta *representa la respuesta HTTP que envía una aplicación Express cuando recibe una solicitud HTTP*

Tipos de respuesta en este proyecto:

Respuesta	Ejemplo
res.json(entityObject)	devuelve el objeto entityObject al cliente como un documento JSON con el código de estado HTTP 200. Por ejemplo: devolverá el objeto del restaurante **res.json**(restaurant) como documento json.
res.json(message)	devuelve una cadena message al cliente como documento JSON con código de estado HTTP 200.
res.status(500).**send**(err)	devuelve el objeto err (normalmente incluyendo algún tipo de mensaje de error) y un código de estado HTTP 500 al cliente

El estado del código HTTP utilizado en este proyecto es:

- **200. Las solicitudes fueron atendidas con éxito.**
- **401. Credenciales equivocadas.**
- **403. Solicitud prohibida (no hay suficientes privilegios).**
- **404. No se encontró el recurso solicitado.**
- **422. Error de validación.**
- **500. Error general.**

MÉTODOS DE CONTROLADORES PARA RESTAURANT

¡PASO PREVIO: Implementar el controlador:!

Importa los modelos necesarios al principio del archivo:

import { Restaurant, Product, ProductCategory, RestaurantCategory } from ('../models')

1. Create (para crear un restaurante) …… req.body

Necesitaremos dos métodos para los modelos: build y save (Model.build y Model.save)

```
const create = async function (req, res) => {

const newRestaurant = Restaurant.build(req.body)

try {

   newRestaurant.userId = 1 // Temporal hasta Lab 3
   const restaurant = await newRestaurant.save()
   res.json(restaurant)

 } catch (err) {

   res.status(500).send(err)

 }

}
```

2. Index (para listar los restaurantes) …… find.All

Los clientes podrán consultar todos los restaurantes.

```
const index = async function (req, res) => {
 try {
  const restaurants = await Restaurant.findAll(
  {
     attributes: { exclude: ['userId'] },
     include:
    {
     model: RestaurantCategory,
     as: 'restaurantCategory'
    },
     order: [[{ model: RestaurantCategory, as: 'restaurantCategory' }, 'name', 'ASC']] (*)

    }                                                                    (*) ó DESC

)
   res.json(restaurants)
 } catch (err) {
   res.status(500).send(err)
 }
}
```

3. Show (Detalles y menú de restaurantes) …… findByPk

Los clientes podrán consultar los datos de los restaurantes y los productos que ofrecen. Por ello, recibirás una identificación del restaurante **req**.params.restaurantId. Puedes usar el método Secuelarize **Model**.findByPk. Ten en cuenta que tendrás que incluir sus productos y su categoría de restaurante. Recuerda que los productos deben ordenarse según el valor del campo de orden. Puedes usar el siguiente fragmento de código para realizar la consulta:

```
const show = async function (req, res) => {
 try {
  const restaurant = await Restaurant. findByPk (req.params.restaurantId, {
   attributes: { exclude: ['userId'] },
   include: [{
      model: Product,
      as: 'products',
      include: { model: ProductCategory, as: 'productCategory' }
   },
   {
      model: RestaurantCategory,
      as: 'restaurantCategory'
   }],
   order: [[{ model: Product, as: 'products' }, 'order', 'ASC']]                    (ó DESC)
  })

  res.json(restaurant)
 } catch (err) {
  res.status(500).send(err)
 }
}
```

4. Update (actualizar) …… Model.update

Usa el método Model.update. En caso de éxito, debes devolver el elemento actualizado del restaurante consultando la base de datos (usando el método findByPk) después de la actualización. Este método sigue los mismos pasos que al crear un restaurante.

```
const update = async function (req, res) => {
 try {
  await Restaurant.update(req.body, {
    where: { id: req.params.restaurantId}
  })
  const updateRestaurant = await Restaurant. findByPk (req.params.restaurantId)
  res.json(updateRestaurant)
 } catch (err) {
  res.status(500).send(err)
 }
}
```

5. Destroy (eliminar) …… Model.destroy

Usa el método **Model**.destroy. Necesitas especificar una cláusula where para eliminar solo el restaurante identificado por **req**.params.restaurantId. destroy devuelve el número de elemento (id) destruido dentro de un mensaje informativo.

```
const destroy = async function (req, res) => {
 try {
  const result = await Restaurant.destroy ( { where: { id: req.params.restaurantId }})

  let message = ‘’
   if (result === 1) {
    message = 'Successfully deleted restaurant id.' + req.params.restaurantId
   } else {
    message = 'Could not delete restaurant.'
   }

  res.json(message)
 } catch (err) {
  res.status(500).send(err)
 }
}
```

EJEMPLO DE CONTROLADOR CON DEFAULT

```
async markDefault (req, res) {
  const { shippingAddressId } = req.params
  const userId = req.user.id
  try {
   const address = await ShippingAddress.findOne({ where: { id: shippingAddressId } })
   await ShippingAddress.update({ isDefault: false }, {
    where: {
     id: { [Op.ne]: shippingAddressId }
    }
   })
   address.isDefault = true
   await address.save()
   return res.status(200).json(address)

  } catch (error) {
   console.error('Error en markDefault:', error)
   return res.status(500).json({ message: 'Internal server error' })
  }
 }
```

7. PRUEBAS Y FUNCIONAMIENTO DEL BACKEND

Paso 1: Ubícate en la carpeta principal del proyecto:

Paso 2: ejecuta: **npm run test:backend**

Si está correcto, todos los ticks (√) deben estar en verde (PASS):

Ejemplo OK:

```
PASS  tests/e2e/restaurants.test.js
  Get all restaurants
    √ There must be more than one restaurant (69 ms)
    √ All restaurants must have an id
    √ All restaurants must have a restaurant category
    √ All restaurants properties must be correctly defined (7 ms)

Test results saved to testResults.csv
Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
```

A. Anexo sobre transacciones

Algunas operaciones de base de datos deben estar incluidas en una transacción de base de datos. Esto es útil cuando se necesita para asegurar que varias operaciones se ejecuten con éxito en la base de datos y, en caso de que alguna de ellas genere una excepción, deshacer todas las operaciones anteriores.

Ejemplo de código fragmentado:

```
const models = require('../models')
const EntityName = models.EntityName

const someFunctionThatNeedsTransaction = async (req, res) => {
  let newEntity = EntityName.build(req.body)
  const transaction = await models.sequelize.transaction() //creates a transaction
  try {
    newEntity = await newEntity.save({transaction}) //use transaction in every operation
    await newEntity.addRelatedEntity(relatedEntityId, { through: { associatedAttribute1: value1,
                                                                   associatedAttributeN: valueN }, transaction })
    await transaction.commit() //confirm all operations
    res.json(newEntity)
  } catch (err) {
    await transaction.rollback() //in case of error, rollback all operations executed in the context of the transaction
    res.status(500).send(err)
  }
}
```

B. Modelos de códigos

1. Validación con Promise.

```
const schedule = await Schedule.findByPk(value)
 if (!schedule) {
  return Promise.reject(new Error('The scheduleId does not exist.'))
 }
 if (schedule.restaurantId !== req.body.restaurantId) {
  return Promise.reject(new Error('The scheduleId does not belong to the given restaurantId.'))
 }
 return Promise.resolve()
}
```

2. Middleware para comprobar que un cliente puede crear una review.

```
const userHasPlacedOrderInRestaurant = async (req, res, next) => {
const userId = req.user.id
const { restaurantId } = req.params
const ordersCount = await Order.count({ where: { userId, restaurantId } })
if (ordersCount === 0) {
return res.status(409).json({ error: 'User cannot review this restaurant without completed orders.'
})
}
next()
}

const checkCustomerHasNotReviewed = async (req, res, next) => {
try {
const { restaurantId } = req.params
const customerId = req.user.id
const existingReview = await Review.findOne({
where: { restaurantId, customerId }
})
if (existingReview) {
return res.status(409).json({ message: 'You have already reviewed this restaurant'
})
}
next()
} catch (error) {
return res.status(500).json({ message: 'An error occurred while checking reviews', error
})
}
```

3. **Cálculo de la media de estrellas en Restaurant.**

```
async getAvgStars () {
    try {
      const reviews = await this.getReviews()
      if (!reviews.length) return
      const totalStars = reviews.reduce((acc, review) => acc + review.stars, 0)
      return totalStars / reviews.length
    } catch (err) {
      return err
    }
  }
}
```

C. Operadores de consulta

```
const { Op } = require('sequelize');

// Operadores comunes
const whereClause = {
  field1: { [Op.gt]: someValue },     // Mayor que
  field2: { [Op.lt]: someValue },     // Menor que
  field3: { [Op.gte]: someValue },    // Mayor o igual que
  field4: { [Op.lte]: someValue },    // Menor o igual que
  field5: { [Op.eq]: someValue },     // Igual a
  field6: { [Op.ne]: someValue },     // No igual a
  field7: { [Op.is]: null },          // Es NULL
  field8: { [Op.not]: null },         // No es NULL
  field9: { [Op.in]: [1, 2, 3] },     // En el array
  field10: { [Op.notIn]: [1, 2, 3] },// No en el array
  field11: { [Op.like]: '%text%' },   // Contiene texto
};
```